```
-- ================================================================
-- مساعد الأعمال الشخصي — مخطط Supabase الكامل
-- الصق هذا الملف كاملًا في: Supabase → SQL Editor → New query → Run
-- آمن لإعادة التشغيل (idempotent).
--
-- المبدأ: كل مشترٍ = عميل مستقل تمامًا.
-- client_id = auth.users.id
-- العزل مفروض داخل قاعدة البيانات (RLS) وليس في كود الواجهة فقط.
-- ================================================================

create extension if not exists vector with schema extensions;
create extension if not exists pgcrypto with schema extensions;
```

```
-- ---------- دوال مساعدة ---------
-
create or replace function
public.set_updated_at()
returns trigger language plpgsql
as $$
begin new.updated_at = now();
return new; end $$;

-- ---------- 1) العملاء ----------
create table if not exists
public.clients (
  id          uuid primary key
references auth.users(id) on
delete cascade,
  full_name   text,
  company     text,
  timezone    text not null
default 'Asia/Riyadh',
  locale      text not null
default 'ar',
  plan        text not null
default 'trial' check (plan in
('trial','active','suspended')),
  trial_ends_at timestamptz
```

```
default (now() + interval '14
days'),
  created_at  timestamptz not
null default now(),
  updated_at  timestamptz not
null default now()
);
drop trigger if exists
trg_clients_updated on
public.clients;
create trigger
trg_clients_updated before update
on public.clients
  for each row execute function
public.set_updated_at();

-- ---------- 2) القنوات المربوطة
(إيميل/X/واتساب/تلغرام) ----------
create table if not exists
public.channels (
  id          uuid primary key
default gen_random_uuid(),
  client_id   uuid not null
references public.clients(id) on
delete cascade,
  kind        text not null check
```

```
(kind in
('whatsapp_qr','whatsapp_cloud','
telegram','x','email')),
  label       text,
  external_id
text,                         --
رقم/معرّف الحساب في المنصة
  status      text not null
default 'pending' check (status
in
('pending','connected','error','d
isabled')),
  created_at  timestamptz not
null default now(),
  updated_at  timestamptz not
null default now(),
  unique (client_id, kind,
external_id)
);
create index if not exists
channels_client_idx on
public.channels(client_id);
create index if not exists
channels_ext_idx on
public.channels(kind,
external_id);
```

```
drop trigger if exists
trg_channels_updated on
public.channels;
create trigger
trg_channels_updated before
update on public.channels
  for each row execute function
public.set_updated_at();

-- أسرار القنوات (توكنات/جلسات): لا سياسات
لا يقرؤها المتصفح أبدًا. =
-- يصل إليها الخادم فقط عبر
SUPABASE_SERVICE_ROLE_KEY. خزّن
القيم مشفّرة من الكود.
create table if not exists
public.channel_secrets (
  channel_id  uuid primary key
references public.channels(id) on
delete cascade,
  secret_enc  text not null,
  updated_at  timestamptz not
null default now()
);

-- ---------- 3) الرسائل ----------
create table if not exists
```

```
public.messages (
  id          uuid primary key
default gen_random_uuid(),
  client_id   uuid not null
references public.clients(id) on
delete cascade,
  channel_id  uuid references
public.channels(id) on delete set
null,
  direction   text not null check
(direction in ('in','out')),
  external_id
text,                       --
معرّف الرسالة في المنصة (لمنع التكرار)
  contact
text,                       --
المرسِل/المستلم
  contact_name text,
  kind        text not null
default 'text' check (kind in
('text','audio','image','document
','other')),
  body
text,                       --
النص أو التفريغ الصوتي
  media_path
```

```
text,                          --
مسار الملف في Storage (إن وُجد)
  summary     text,
  importance  smallint check
(importance between 1 and 5),
  read_by_client boolean not null
default false,
  sent_at     timestamptz not
null default now(),
  created_at  timestamptz not
null default now(),
  unique (client_id, channel_id,
external_id)
);
create index if not exists
messages_client_time_idx on
public.messages(client_id,
sent_at desc);
create index if not exists
messages_important_idx on
public.messages(client_id,
importance desc, sent_at desc);

-- ---------- 4) المهام (أوامر العمل /
ما يُطلب من المساعد) ----------
create table if not exists
```

```
public.tasks (
  id          uuid primary key
default gen_random_uuid(),
  client_id   uuid not null
references public.clients(id) on
delete cascade,
  title       text not null,
  details     text,
  assignee
text,                         --
اسم شخص من جهات اتصاله (نص حر، بلا
(حسابات
  due_at      timestamptz,
  status      text not null
default 'open' check (status in
('open','in_progress','done','can
celled')),
  source_message_id uuid
references public.messages(id) on
delete set null,
  created_at  timestamptz not
null default now(),
  updated_at  timestamptz not
null default now()
);
create index if not exists
```

```
tasks_client_idx on
public.tasks(client_id, status,
due_at);
drop trigger if exists
trg_tasks_updated on
public.tasks;
create trigger trg_tasks_updated
before update on public.tasks
  for each row execute function
public.set_updated_at();

-- ---------- 5) جدار الصلاحيات -----
-----
-- كتالوج الأفعال التي يعرفها النظام (يديره
المالك/الخادم فقط)
create table if not exists
public.action_catalog (
  action      text primary
key,           -- مثل:
message.send
  label_ar    text not null,
  risk        text not null check
(risk in
('read','write','device')),
  default_mode text not null
check (default_mode in
```

```
('auto','confirm','deny'))
);
insert into
public.action_catalog(action,labe
l_ar,risk,default_mode) values
  ('message.read',      'قراءة
الرسائل وتلخيصها',            'read',
'auto'),
  ('message.send',      'إرسال رسالة
نيابة عني',              'write',
'confirm'),
  ('email.read',        'قراءة
البريد',
'read',  'auto'),
  ('email.send',        'إرسال بريد
نيابة عني',              'write',
'confirm'),
  ('task.create',       'إنشاء مهام
وتوزيعها',               'write',
'auto'),
  ('task.assign_send',  'إرسال أمر
عمل لشخص بالاسم',          'write',
'confirm'),
  ('voice.transcribe',  'تحويل
الصوت إلى نص',
'read',  'auto'),
```

```
  ('device.open_app',   'فتح تطبيق
على جهازي',
'device','confirm'),
  ('device.tap',        'النقر والتنقل
داخل تطبيق',
'device','confirm')
on conflict (action) do update
  set label_ar =
excluded.label_ar, risk =
excluded.risk;   -- لا نغيّر
default_mode لو عدّله المالك

-- صلاحيات كل عميل: القيمة الافتراضية للغائب
هي «منع» (الجدار يفشل مغلقًا)
create table if not exists
public.permissions (
  client_id   uuid not null
references public.clients(id) on
delete cascade,
  action      text not null
references
public.action_catalog(action) on
delete cascade,
  mode        text not null check
(mode in
('auto','confirm','deny')),
```

```
  scope       jsonb not null
default '{}'::jsonb,   -- قيود
اختيارية: جهات مسموحة، تطبيقات مسموحة...
  updated_at  timestamptz not
null default now(),
  primary key (client_id, action)
);
drop trigger if exists
trg_permissions_updated on
public.permissions;
create trigger
trg_permissions_updated before
update on public.permissions
  for each row execute function
public.set_updated_at();

-- دالة يستدعيها الخادم قبل أي فعل: ترجع
auto / confirm / deny
create or replace function
public.action_mode(p_client uuid,
p_action text)
returns text language sql stable
security invoker set search_path
= public as $$
  select coalesce((select mode
from public.permissions where
```

```
client_id = p_client and action =
p_action), 'deny')
$$;

-- طابور الأفعال التي تنتظر تأكيد العميل
create table if not exists
public.pending_actions (
  id          uuid primary key
default gen_random_uuid(),
  client_id   uuid not null
references public.clients(id) on
delete cascade,
  action      text not null
references
public.action_catalog(action),
  payload     jsonb not
null,             -- ما سيُنفَّذ بالضبط
(المستلم، النص، التطبيق...)
  status      text not null
default 'waiting' check (status
in
('waiting','approved','rejected',
'expired','done','failed')),
  created_at  timestamptz not
null default now(),
  decided_at  timestamptz,
```

```
  expires_at  timestamptz not
null default (now() + interval '1
day')
);
create index if not exists
pending_client_idx on
public.pending_actions(client_id,
status, created_at desc);

-- سجل تدقيق: كل ما فعله المساعد (للقراءة
فقط من جهة العميل)
create table if not exists
public.action_log (
  id          bigint generated
always as identity primary key,
  client_id   uuid not null
references public.clients(id) on
delete cascade,
  action      text not null,
  mode_applied text not null,
  payload     jsonb,
  result      text,
  created_at  timestamptz not
null default now()
);
create index if not exists
```

```
action_log_client_idx on
public.action_log(client_id,
created_at desc);

-- ---------- 6) ذاكرة المساعد (للتعلّم)
----------
create table if not exists
public.memory_chunks (
  id          uuid primary key
default gen_random_uuid(),
  client_id   uuid not null
references public.clients(id) on
delete cascade,
  kind        text not null
default 'fact' check (kind in
('fact','preference','correction'
,'summary')),
  content     text not null,
  embedding
extensions.vector(1536),    --
text-embedding-3-small
  created_at  timestamptz not
null default now()
);
create index if not exists
memory_client_idx on
```

```
public.memory_chunks(client_id);
create index if not exists
memory_vec_idx on
public.memory_chunks
  using hnsw (embedding
extensions.vector_cosine_ops);

-- بحث دلالي ضمن ذاكرة العميل نفسه فقط
(يحترم RLS لأنه security invoker)
create or replace function
public.match_memory(p_embedding
extensions.vector(1536), p_limit
int default 8)
returns table (id uuid, content
text, kind text, similarity
float)
language sql stable security
invoker set search_path = public,
extensions as $$
  select m.id, m.content, m.kind,
1 - (m.embedding <=> p_embedding)
as similarity
  from public.memory_chunks m
  where m.client_id = auth.uid()
and m.embedding is not null
  order by m.embedding <=>
```

```
p_embedding
  limit p_limit
$$;

-- ---------- 7) إنشاء العميل تلقائيًا عند
التسجيل ----------
create or replace function
public.handle_new_user()
returns trigger language plpgsql
security definer set search_path
= public as $$
begin
  insert into public.clients(id,
full_name)
  values (new.id,
coalesce(new.raw_user_meta_data-
>>'full_name', ''))
  on conflict (id) do nothing;
  insert into
public.permissions(client_id,
action, mode)
  select new.id, action,
default_mode from
public.action_catalog
  on conflict do nothing;
  return new;
```

```
end $$;
drop trigger if exists
on_auth_user_created on
auth.users;
create trigger
on_auth_user_created after insert
on auth.users
  for each row execute function
public.handle_new_user();

-- ---------- 8) RLS: كل عميل لا يرى
إلا بياناته ----------
alter table
public.clients         enable row
level security;
alter table
public.channels        enable row
level security;
alter table
public.channel_secrets enable row
level security;   -- بلا سياسات عمدًا
alter table
public.messages        enable row
level security;
alter table
public.tasks           enable row
```

```
level security;
alter table
public.action_catalog  enable row
level security;
alter table
public.permissions     enable row
level security;
alter table
public.pending_actions enable row
level security;
alter table
public.action_log      enable row
level security;
alter table
public.memory_chunks   enable row
level security;

drop policy if exists
clients_self on public.clients;
create policy clients_self on
public.clients for select to
authenticated using (id = (select
auth.uid()));
drop policy if exists
clients_self_upd on
public.clients;
```

```
create policy clients_self_upd on
public.clients for update to
authenticated
  using (id = (select
auth.uid())) with check (id =
(select auth.uid()));

-- القنوات: يقرأ ويحذف عميلها؛ الإنشاء/التعديل عبر الخادم
drop policy if exists
channels_read on public.channels;
create policy channels_read on
public.channels for select to
authenticated using (client_id =
(select auth.uid()));
drop policy if exists
channels_delete on
public.channels;
create policy channels_delete on
public.channels for delete to
authenticated using (client_id =
(select auth.uid()));

-- الرسائل: قراءة + تعليم كمقروءة؛ الكتابة من الخادم
drop policy if exists
```

```
messages_read on public.messages;
create policy messages_read on
public.messages for select to
authenticated using (client_id =
(select auth.uid()));
drop policy if exists
messages_mark on public.messages;
create policy messages_mark on
public.messages for update to
authenticated
  using (client_id = (select
auth.uid())) with check
(client_id = (select
auth.uid()));

-- المهام: تحكم كامل لصاحبها
drop policy if exists tasks_all
on public.tasks;
create policy tasks_all on
public.tasks for all to
authenticated
  using (client_id = (select
auth.uid())) with check
(client_id = (select
auth.uid()));
```

```
-- الكتالوج: قراءة فقط
drop policy if exists
catalog_read on
public.action_catalog;
create policy catalog_read on
public.action_catalog for select
to authenticated using (true);

-- الصلاحيات: المالك وحده يغيّرها (لا يستطيع
المساعد/الخادم تعديلها بدون جلسته)
drop policy if exists perms_read
on public.permissions;
create policy perms_read on
public.permissions for select to
authenticated using (client_id =
(select auth.uid()));
drop policy if exists perms_upd
on public.permissions;
create policy perms_upd on
public.permissions for update to
authenticated
  using (client_id = (select
auth.uid())) with check
(client_id = (select
auth.uid()));
```

```
-- طابور التأكيد: يقرأ ويقرر صاحبه (يوافق/
يرفض فقط)
drop policy if exists
pending_read on
public.pending_actions;
create policy pending_read on
public.pending_actions for select
to authenticated using (client_id
= (select auth.uid()));
drop policy if exists
pending_decide on
public.pending_actions;
create policy pending_decide on
public.pending_actions for update
to authenticated
  using (client_id = (select
auth.uid()) and status =
'waiting')
  with check (client_id = (select
auth.uid()) and status in
('approved','rejected'));

-- سجل التدقيق: قراءة فقط
drop policy if exists log_read on
public.action_log;
create policy log_read on
```

```
public.action_log for select to
authenticated using (client_id =
(select auth.uid()));

-- الذاكرة: يقرأ ويحذف صاحبها (حق النسيان)
drop policy if exists memory_read
on public.memory_chunks;
create policy memory_read on
public.memory_chunks for select
to authenticated using (client_id
= (select auth.uid()));
drop policy if exists
memory_delete on
public.memory_chunks;
create policy memory_delete on
public.memory_chunks for delete
to authenticated using (client_id
= (select auth.uid()));

-- من أي جدول (anon) منع الدور المجهول
revoke all on all tables in
schema public from anon;
revoke all on all functions in
schema public from anon;

-- ---------- 9) التخزين (الصوت والصور
```

```
(والمستندات ----------
insert into storage.buckets (id,
name, public, file_size_limit)
values ('media', 'media', false,
26214400)          -- خاص، حد
25MB للملف
on conflict (id) do update set
public = false, file_size_limit =
excluded.file_size_limit;

-- المسار الإلزامي: <client_id>/<اسم
الملف>
drop policy if exists media_read
on storage.objects;
create policy media_read on
storage.objects for select to
authenticated
  using (bucket_id = 'media' and
(storage.foldername(name))[1] =
(select auth.uid())::text);
drop policy if exists media_write
on storage.objects;
create policy media_write on
storage.objects for insert to
authenticated
  with check (bucket_id = 'media'
```

```
and (storage.foldername(name))[1]
= (select auth.uid())::text);
drop policy if exists
media_delete on storage.objects;
create policy media_delete on
storage.objects for delete to
authenticated
  using (bucket_id = 'media' and
(storage.foldername(name))[1] =
(select auth.uid())::text);

-- ---------- 10) تنظيف تلقائي
(اختياري: فعّل pg_cron من Database →
Extensions) ----------
-- select cron.schedule('expire-
pending','*/15 * * * *',
--   $$update
public.pending_actions set
status='expired' where
status='waiting' and expires_at <
now()$$);
-- select cron.schedule('purge-
media-90d','0 3 * * *',
--   $$update public.messages set
media_path=null where media_path
is not null and created_at <
```

```
now() - interval '90 days'$$);
-- (يتم من Storage حذف الملفات نفسها من
الخادم؛ يوفّر مساحة الخطة المجانية)
```